LibMapaStatic

Librería de mapas por teselas para Qt — arquitectura, clases y herramientas

---

Documentación técnica · Qt 5.14 / 5.15 / 6.x (MinGW · MSVC · GCC)
Núcleo de teselas RMaps/XYZ · Widget de mapa (QCustomPlot) · Herramientas de línea de comandos y con mapa

Índice

1. Visión general — qué es y qué resuelve
2. Arquitectura por capas
3. Tipos y modelos de datos
4. Módulo **geo** — geodesia y teselas
5. Módulo **db** — SQLite
6. Módulo **tiles** — motor de teselas (6b: **vector**, MBTiles vectoriales)
7. Módulo **widget** — el mapa en pantalla
8. Módulo **io** — ficheros .geo
9. **libtelemetria** — decodificador de NMEA 0183 y AIS (biblioteca aparte)
0. Flujos de interacción entre objetos
1. Codificación de teselas (la matemática)
2. Herramientas desarrolladas
3. Motor de descarga `TileFiller` y auto-freno
4. Decisiones de diseño e historia
5. Cómo compilar y usar

1 · Visión general

LibMapaStatic es una librería en C++/Qt para mostrar mapas basados en **teselas** (imágenes de 256×256 px organizadas por zoom/x/y) guardadas en bases de datos **SQLite** del tipo *RMaps/XYZ*, y para dibujar encima entidades vectoriales (puntos, líneas, polígonos) y **objetivos móviles** en tiempo real. Nace de reescribir una aplicación previa («EstacionTerrena») separando con claridad el **núcleo** (datos, geodesia, teselas) del **dibujo**, de modo que el núcleo se pueda probar sin ventanas.

Qué hace, en una frase por área:

- **Teselas base:** abre una o varias BD de teselas (OSM, satélite…), las cachea y las pinta fluido según navegas.
- **Entidades:** dibuja y edita puntos/líneas/polígonos en capas, con guardado/carga en SQLite y carga de ficheros `.geo`.
- **Objetivos móviles:** móviles con traza y etiqueta, actualizables varias veces por segundo, en una capa que se repinta sola. **Agnósticos del dominio**: cada objetivo lleva un `kind` y un juego de `attributes` libres (AIS, ADS-B, telemetría…) que la app rellena y la librería no interpreta, así el mismo modelo sirve a seguimiento naval, aéreo o de UAVs. La **simbología va por hooks**: la app registra un proveedor (`setTargetSymbolProvider`) que elige el icono de cada objetivo según su `kind`/estado; la librería lo coloca y lo gira por el rumbo, y sin proveedor usa un galón por defecto. **Escala a miles** (ADS-B regional): culling por vista, **nivel de detalle** (por encima de un presupuesto se apagan etiquetas y trazas, que a esa densidad son ilegibles), **declutter** de etiquetas (una por celda) y decimación de la traza.
- **Herramientas:** sondar una BD (`probe_db`), rasterizar vectores a teselas (`geo_to_tiles`), **descargar/rellenar** teselas de una fuente XYZ (`fill_tiles` consola y `fill_map` con mapa), elevación (`fill_hgt` descarga SRTM, `dem_to_db` lo empaqueta), **hornear relieve sombreado** del DEM a teselas (`dem_to_hillshade`, gris o `--colored` con batimetría, offline) y **generar curvas de nivel** del DEM como capa vectorial (`dem_to_contours`). Scripts aparte convierten datos de **OpenStreetMap** a capas de entidades: la **costa** (`osm_costa_a_sqlitedb.py`, máscara de tierra para `--water-mask` y overlay de costa) y las **divisiones administrativas** (`osm_divisiones_a_sqlitedb.py`, una entidad con nombre por provincia/municipio). La librería lee también **MBTiles** ráster y **vectoriales** (pbf/MVT, p. ej. OpenMapTiles; los pinta con un estilo, módulo `vector`) (`probe_db` los detecta solos, o basta `id` + `filePath` en `datasets.json`) y elevación en **GeoTIFF**. Una capa base se puede **añadir en caliente** (`MapWidget::addBaseLayer` → `TileService::addDataset` + `TileLoader::addDataset`, que no vacía las fuentes ya abiertas). La **búsqueda de lugares** (`MapWidget::searchPlaces` / `goToPlace`) consulta un índice local `lugares.sqlitedb` (`PlaceIndex`, `"places"` en el manifiesto) generado por `osm_lugares_a_sqlitedb.py`.

El diseño evita los defectos del código original: jerarquías con punteros crudos y `void*`, macros de zoom dispersas, y una clase de dibujo que exponía 300 KB de cabecera de QCustomPlot a todo el que la usara. Aquí la identidad de cada entidad es un **entero**, el mapeo de zoom vive en **datos** (`datasets.json`), y QCustomPlot queda **oculto** tras un PIMPL.

2 · Arquitectura por capas

El código se organiza en dos bibliotecas estáticas y un conjunto de aplicaciones. El **núcleo** no depende de Widgets ni de QCustomPlot; el **widget** añade el mapa visual. Las herramientas usan una u otra según necesiten pantalla.

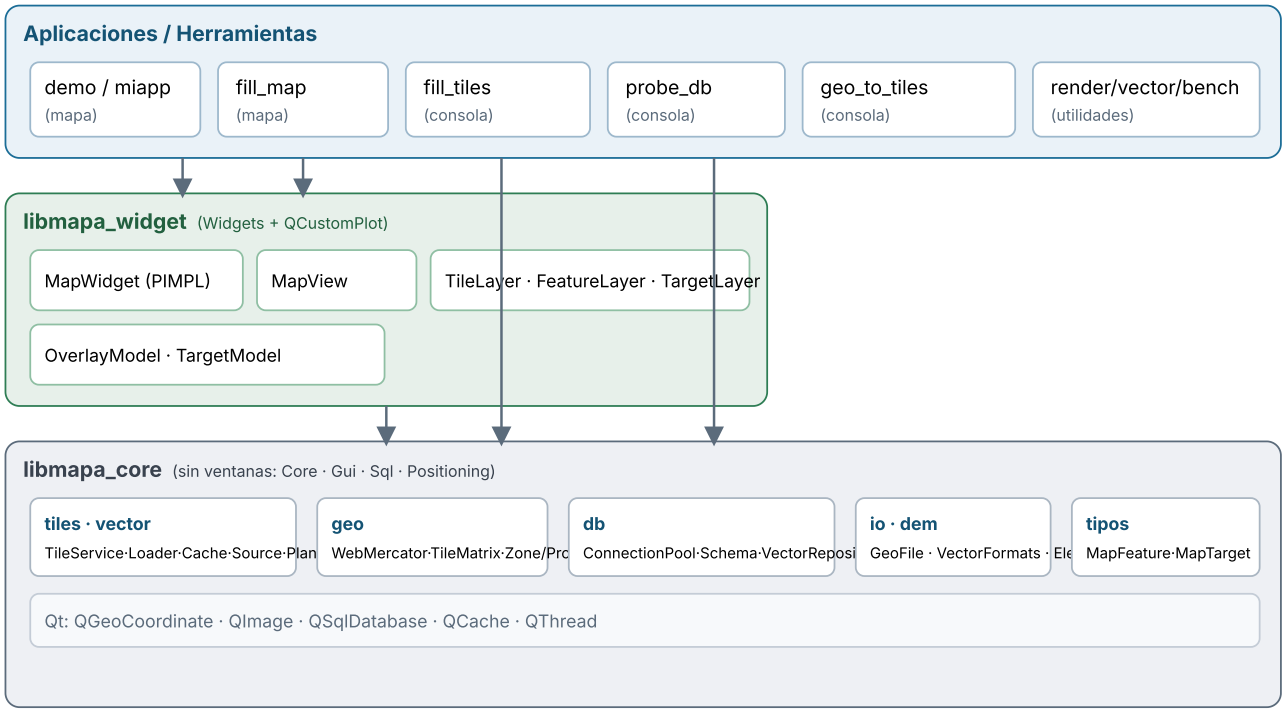


Figura 1 — Capas: las apps usan el widget o directamente el núcleo; el widget se apoya en el núcleo; el núcleo solo depende de Qt.

Motivo de la separación: el núcleo (teselas, geodesia, BD) se compila y prueba **sin QCustomPlot ni Widgets**. Así 14 de los 17 tests automáticos validan la lógica sin QCustomPlot ni ventanas, y quien solo quiera leer teselas no arrastra el motor de dibujo.

3 · Tipos y modelos de datos

Antes que las clases con lógica, conviene conocer los **tipos de valor** que se pasan por toda la librería. Son *structs* simples (sin herencia), identificados por un entero cuando hace falta.

Tipo	Rol
`TileKey`	Identidad de una tesela: `{z, x, y}` lógicos. Se usa como clave del caché (con `qHash`).
`TileDataset`	Todo lo que se sabe de una BD de teselas: ruta, tabla, mapeo de zoom (`zFactor/zOffset`), esquema (XYZ/TMS), columnas, `hasSColumn/sValue`, min/max/recommended zoom, relleno típico. Se lee de `datasets.json`.
`MapFeature`	Entidad vectorial: geometría multi-parte (puntos/líneas/polígonos), estilo (`FeatureStyle`), etiqueta y «tipo de dominio». Identidad = `qint64`. La etiqueta puede **desplazarse** (`FeatureStyle::labelOffsetPx`, con línea guía) y anclarse a un vértice concreto (`labelAnchorVertex`); igual en `MapTarget::labelOffsetPx`. `FeatureStyle::labelAnchor` (`LabelAnchor`: `Auto`, que es el cálculo de siempre y el valor por defecto, y nueve posiciones) cuelga la caja entera de un lado del anclaje, con el desplazamiento sumado encima, y `labelFontSize` le da fuente propia (0 = la del mapa); ambos se guardan (esquema SQLite v4). `MapWidget::featureLabelRect` da la caja en píxeles.
`Círculos, arcos y partes`	`GeometryKind::Circle` y `Arc` (`radiusMeters`, `arcStartDeg`, `arcEndDeg`; esquema SQLite v5) son geodésicos: `outlines()` da el contorno muestreado, `storedParts()` el centro y `toPlain()` el polígono o la polilínea de los exportadores. `MapWidget::addFeatures` da de alta en lote (una señal, un paso de deshacer), `featureTypeCounts` / `clearFeaturesOfType` cuentan y borran por tipo y `setFeatureTypeVisible` lo oculta como filtro de la vista. `MapFeature::partsInfo` (`PartInfo`: nombre, color, ancla; tabla `entidad_parte`, esquema v6) es paralelo a `parts`; `MapHit::subIndex` y `featureAt(pixel, tol, &sub)` dicen qué parte hay bajo el pixel. `MapFeature::originTargetId` y `originFeatureId` dan dueño a un círculo o sector (como `RangeRings::targetId`): la librería lo mantiene sobre su dueño y lo quita con él (esquema v8). `RangeRings` rotula distancias y marcaciones por separado (`distanceLabels`, `bearingLabels`), translúcidos (`labelOpacity`) y destacados bajo el ratón. `MapWidget::setTargetLabelContent` cambia el rótulo por segmentos de un objetivo ya creado sin perder su traza. Un `Arc` con `arcSector` es un sector (cuña rellena, un área; esquema v7) y `roundContains` dice si un punto cae dentro. 5.000 puntos como una entidad con partes ocupan menos de la mitad que 5.000 entidades.
`MapTarget`	Objetivo móvil: posición, rumbo, velocidad, color, etiqueta, visibilidad de traza, más `kind` y un `QVariantMap attributes` libre para datos de dominio (AIS/ADS-B/telemetría) que la librería no interpreta. Pensado para tiempo real y agnóstico del dominio.

`LayerInfo / BaseLayerInfo`	Descriptor de una capa (de entidades / base) para poblar menús.
`GeoPath`	Un trazado leído de un fichero `.geo` (lista de coordenadas + si cierra).
`Measurement`	Resultado de una medición sobre el mapa (distancia y azimut).

Regla de diseño: la identidad es un entero que devuelve la librería al añadir el objeto, no un puntero a una jerarquía. Esto elimina los `QList<void*>` y castes del código original.

4 · Módulo `geo` — geodesia y teselas

Convierte entre coordenadas geográficas (lat/lon) y el sistema de teselas Web Mercator. Es matemática pura, sin estado ni Qt más allá de `QGeoCoordinate`.

`WebMercator` geo/WebMercator.h

Función	Qué hace
`forward(coord) → QPointF`	Lat/lon → metros Web Mercator (EPSG:3857).
`inverse(x,y) → QGeoCoordinate`	Metros → lat/lon (la usa `geo_to_tiles` al leer `.xyz`).
`clampLatitude(lat)`	Acota la latitud al máximo de Mercator (~±85,05°) para que la proyección no se dispare.

`Coordinates` include/libmapa/Coordinates.h

Coordenadas en los formatos de trabajo: decimal, grados-minutos-segundos, grados-minutos decimales, **UTM** y **MGRS** (WGS84). UTM con las series de Krüger de orden 6 (Karney 2011): coincide con PROJ al milímetro; MGRS idéntico a GeoTrans (NGA). Incluye las excepciones de zona de Noruega (32V) y Svalbard (31X–37X). Cobertura UTM: 80°S–84°N (los polos, UPS, no). `tst_coordinates` compara con referencias de PROJ 9.8 y de la librería *mgrs*.

Función	Qué hace
`toUtm(coord, zonaForzada) / fromUtm`	Geográfica ↔ UTM (zona, banda, hemisferio, este, norte). La zona forzada sirve para las cuadrículas que cruzan un límite.
`toMgrs(coord, precisión) / fromMgrs`	MGRS de 10 km a 1 m; los dígitos se truncan (esquina suroeste de la celda), como manda la norma.
`formatCoordinate / parseCoordinate`	Texto en cualquier formato, y lectura de lo que escribe una persona (MGRS, UTM, decimal o sexagesimal con hemisferio, «O» = oeste).
`polygonArea / pathLength`	Superficie (m²) de un polígono en la esfera autálica con **latitudes autálicas** (un trapecio de meridianos y paralelos sale exacto en WGS84) y longitud de un trazado o perímetro (m).

`MapDecor` geo/MapDecor.h

Cálculos puros de la **escala gráfica** y la **cuadrícula** (sin dibujo; `tst_mapdecor`).

Función	Qué hace
`scaleBar(m/px, maxPx, unidades)`	La barra más larga con valor redondo 1-2-5×10ⁿ que cabe; en metros/km o en millas náuticas (redondeo en millas).
`graticuleStep(tramo°, maxLíneas)`	Paso «de carta» (90°…1°, 30'…1', 30"…1") más fino que no supera las líneas pedidas.
`formatLatitude / formatLongitude`	Rótulo sexagesimal con la precisión del paso (23°N, 23°30'N, 23°30'15"N); longitud normalizada a (−180, 180].

`ZoneMonitor` geo/ZoneMonitor.h

Lógica de las **alertas de zona**: qué objetivos están dentro de qué polígonos vigilados y cuándo entran o salen. Sin widget ni señales (se prueba sin ventanas, `tst_zonemonitor`); `MapWidget` le

pasa los cambios y convierte sus transiciones en señales. Prueba de pertenencia en **Web Mercator** (lados rectos como se dibujan: «dentro» es lo que se ve dentro), cualquier anillo de una zona multi-parte, con descarte previo por caja envolvente: O(zonas) por actualización.

Función	Qué hace
`setZone(id, anillos) → transiciones`	Alta o cambio de forma; reevalúa todos los objetivos. Una zona nueva avisa de los que ya estaban dentro.
`updateTarget(id, pos) → transiciones`	Movimiento de un objetivo; posición inválida = sale de todas.
`removeZone · removeTarget · clearTargets`	Olvidan sin generar salidas (borrar no es «salir»).
`targetsIn · zonesOf · isInside`	Consultas del estado actual.

`ProximityMonitor` geo/ProximityMonitor.h · público: Proximity.h

Lógica de las **alertas de proximidad (CPA/TCPA)**: qué pares de móviles van a pasar demasiado cerca. `computeCpa` (público) calcula el punto de máximo acercamiento en un plano tangente local suponiendo rumbo y velocidad constantes: CPA (distancia mínima), TCPA (tiempo hasta ella) y dónde estará cada uno. Un par está en alerta si **ya** está a menos de la distancia, o si el CPA cae por debajo de ella dentro del horizonte (0 ≤ TCPA ≤ horizonte). Antes de comparar dos móviles, el que informó antes se lleva **por estima** a la hora del otro. Coste O(móviles) por actualización, con un descarte barato: la distancia actual menos lo máximo que pueden acercarse en el horizonte. Sin criterios, no calcula nada (`tst_proximity` : 2000 móviles cuadran con la fuerza bruta).

Función	Qué hace
`setCriteria(distancia, horizonte) → transiciones`	Activa, cambia o apaga (distancia 0) y reevalúa todos los pares.
`updateTrack(id, {pos, vel, rumbo, hora}) → transiciones`	Reevalúa los pares de ese móvil; posición inválida = sale de sus alertas.
`removeTrack · clear`	Olvidan sin transiciones (borrar no es «alejarse»).
`alerts · isAlerted · cpa · evaluate`	Pares en alerta con su CPA; CPA de dos móviles cualesquiera.

`TileMatrix` geo/TileMatrix.h

El corazón geométrico: relaciona lat/lon con índices de tesela a cada zoom. Todo *static*.

Función	Qué hace
`tilesPerSide(z)`	`2^z` : cuántas teselas por lado tiene el mundo a ese zoom.
`longitudeToTileX / latitudeToTileY`	Coordenada → índice de tesela (fraccionario) en X / Y.
`tileXToLongitude / tileYToLatitude`	La inversa: índice de tesela → coordenada del borde.
`tileAt(coord, z) → TileKey`	Qué tesela contiene una coordenada a un zoom.

`tileBounds / tileNorthWest / tileSouthEast`	Rectángulo geográfico que cubre una tesela.
`rangeFor(nw, se, z, margen)`	Rango de teselas `[xMin..xMax]×[yMin..yMax]` que cubre un rectángulo — la base para pedir el viewport.
`toStorageY / fromStorageY(y,z,esquema)`	Convierte la Y entre **XYZ** (origen arriba) y **TMS** (origen abajo). Involutiva.
`latitudeToAxisY / axisYToLatitude`	Lat ↔ «grados de Mercator» del eje lineal de QCustomPlot (imprescindible para overlays).
`bestZoomFor(span, anchoPx)`	Elige el zoom natural para un ancho de mundo visible.

`TileKey` tiles/TileKey.h

Struct `{z,x,y}` con `operator==` y `qHash` para servir de clave en el caché. En Qt 6 el `qHash` se adapta (guarda de versión legítima).

5 · Módulo db — SQLite

SqliteConnectionPool db/SqliteConnectionPool.h

Da a cada hilo su propia conexión SQLite (Qt exige que una `QSqlDatabase` no se comparta entre hilos). Reutiliza conexiones por (fichero, hilo) y las nombra de forma única.

Función	Qué hace
`connectionFor(id, ruta)`	Devuelve/crea la conexión para ese fichero en el hilo actual.
`openConnectionCount()`	Cuántas conexiones vivas hay (diagnóstico y tests).

Transaction db/Transaction.h

Envoltura RAII de `BEGIN/COMMIT/ROLLBACK`: `commit()` confirma; si el objeto muere sin confirmar, hace *rollback*. Evita transacciones a medias.

Schema db/Schema.h

Crea/verifica el esquema de la BD de **entidades** (capas, geometrías, estilos) que usa `VectorRepository`. Versionado y migrado por pasos acumulativos (`kCurrentVersion`): **v2** esquema base; **v3** añade el desplazamiento de la etiqueta (`label_dx/label_dy`) con `ALTER TABLE`, de modo que una BD anterior (incluida una máscara generada por script) se abre y se migra sola.

VectorRepository db/VectorRepository.h · QObject

Persiste entidades vectoriales y capas en SQLite: guardar/cargar todo el mapa de entidades. Emite `errorOccurred`. Lo usa `MapWidget` en `saveFeaturesTo()/loadFeaturesFrom()`. Con `MapConfig.featuresDbFile` (o `setFeaturesDbFile()`) el widget hace **persistencia automática**: carga al abrir y guarda solo lo dibujado/editado/borrado con un **antirebote** (~500 ms) que agrupa las ráfagas — volcado completo, reutilizando `saveFeaturesTo`. Los objetivos móviles no se persisten. Con un **paquete** y `featuresDbFile` vacío, se usa por defecto el fichero de entidades del propio paquete. Para un **guardado explícito** (cargar y guardar solo a mano), pon `MapConfig.autosaveFeatures = false`: entonces no se toca el fichero del paquete ni se autoguarda nada aunque se abra un paquete (lo usa así la app `demo`).

6 · Módulo `tiles` — motor de teselas

Es el subsistema más grande: abrir BD de teselas, decidir qué pedir, leerlas en un hilo aparte, cachearlas y entregar un «plan de dibujo». Todo alrededor del `TileDataset`, que describe cada BD.

`TileDataset` tiles/TileDataset.h

Descriptor de una BD. Sustituye a los `#define` invertidos y parches de zoom del código original. Métodos clave:

Función	Qué hace
`storedZ(zLógico)`	`zFactor·z + zOffset` : el valor que va en la columna `z` de la BD.
`logicalZ(zGuardado)`	La inversa.
`effectiveBaseZoom()`	Nivel de fondo garantizado, acotado al rango real.
`zoomInRange / zoomRecommended`	Si un zoom existe / si conviene ofrecerlo en la interfaz.
`fromJson / toJson`	Lee/escribe el descriptor en `datasets.json` .

`ITileSource` / `RMapsTileSource` tiles/RMapsTileSource.h

`ITileSource` es la interfaz «dame la imagen de esta `TileKey`». `RMapsTileSource` la implementa sobre una BD SQLite RMaps/XYZ: traduce el `TileKey` lógico a la fila real (con `storedZ` , la Y del esquema y la columna `s` si existe) y devuelve el BLOB de imagen.

Función	Qué hace
`open()`	Abre la BD y prepara las consultas (resuelve nombres de columna reales).
`fetch(key)`	Una tesela: `SELECT image WHERE z=storedZ AND x AND y [AND s]` .
`fetchRange(z, rango)`	Muchas teselas de un golpe (un viewport).
`availableInRange(...)`	Qué teselas existen (sin traer la imagen), para saber huecos.

`TileCache` tiles/TileCache.h

Caché LRU de imágenes ya decodificadas (basada en `QCache`). Guarda el coste en **KiB** (no bytes, para no desbordar el `int` con 2 GiB), tiene un área **anclada** (el nivel de fondo, que no se expulsa) y una lista de teselas «que no existen» para no volver a pedirlas.

Función	Qué hace
`take(key)`	Copia (compartida) de la imagen si está; si no, cuenta un fallo.
`insert / insertImage`	Mete una tesela (decodificando o ya decodificada), con o sin anclar.
`markMissing / isKnownMissing`	Marca/consulta que una tesela no existe en la BD.
`countCached(...)`	Cuántas del rango están «resueltas» (en caché o marcadas ausentes).
`clear() / setMaxBytes()`	Vaciar (p. ej. tras `reloadBaseLayer`) / cambiar el presupuesto.

`TileLoader` tiles/TileLoader.h · QObject (hilo)

Lee teselas **en un hilo aparte** para no bloquear la interfaz. Recibe `TileRequest` (un rango + zoom + id), consulta la `ITileSource` , e informa con `LoadedTile` . El id de petición permite descartar respuestas viejas cuando el usuario ya se movió.

`TilePlanner` tiles/TilePlanner.h

Calcula el **plan de dibujo**: qué teselas y con qué escala hay que pintar para cubrir el viewport, incluida la «escalera de respaldo» (usar teselas de zoom vecino mientras llega el detalle). Devuelve `TileDrawItem`s.

`TileService` tiles/TileService.h · QObject

La **fachada** del módulo: orquesta dataset activo + caché + loader + planner. Es lo que usa el widget.

Función	Qué hace
`start(datasets, cacheMiB)`	Arranca con la lista de datasets y el presupuesto de caché.
`loadDatasets(json) [static]`	Lee `datasets.json` → lista de `TileDataset`.
`setActiveDataset(id)`	Cambia de capa base (osm↔satelital↔…).
`requestViewport(nw, se, z, margen)`	Pide al loader las teselas visibles (con antirebote).
`planFor(nw, se, z)`	Plan de dibujo con lo que haya en caché.
`cache()`	Acceso al `TileCache` (p. ej. para vaciarlo).
`signal tilesReady(id, z, nuevas)`	Llegaron teselas nuevas → repintar.

`TileDatasetProbe` tiles/TileDatasetProbe.h

Sonda: abre una BD «a ciegas» y **deduce** su configuración (mapeo de zoom, esquema, relleno, bbox) analizando qué hay en la tabla. Es el cerebro de `probe_db`. Devuelve `ProbeResult` con `ZoomLevelStats` por nivel.

Reconoce además los **MBTiles** (ráster PNG/JPG/WebP): localiza la tabla *o vista* `tiles`, y al detectar la tabla `metadata` fija por especificación el eje Y en **TMS** y el zoom estándar (`storedZ = logicalZ`), tomando la extensión de `metadata.bounds` — así se sondan **sin** `--ref-bbox`. Los MBTiles vectoriales (MVT/PBF) se marcan `vector` (ver 6b).

MBTiles sin sondar src/tiles/MBTiles.h: no hay un lector aparte —un MBTiles es una base con otras columnas (`zoom_level`, `tile_column`, `tile_row`, `tile_data`), TMS y sin `s`, justo lo que `RMapsTileSource` ya lee parametrizado por `TileDataset`—. `MBTiles::applyLayout` pone esa disposición (`TileDataset::mbtiles`) cuando el `filePath` acaba en `.mbtiles` o la entrada lleva `"format":"mbtiles"`, y `MBTiles::describe` completa del propio fichero lo que el JSON no diga (zooms, lado de tesela, nombre, esquema), también al cargar un paquete de datos. Lo escrito a mano manda, y las entradas ya existentes no cambian.

6b · Módulo `vector` — MBTiles vectoriales (pbf / MVT)

Un MBTiles con `format = pbf` (Mapbox Vector Tile 2.1, normalmente en gzip: OpenMapTiles, planetiler, tippecanoe) entra como cualquier otra capa base. La idea es que el resto del motor **no se entere**: el `TileLoader`, en su hilo, en vez de decodificar un PNG **pinta** la tesela vectorial a una `QImage` de `tileSize` (256 px) con un estilo, y entrega `LoadedTile` como siempre. Caché, escalera de respaldo, cobertura, `addBaseLayer`, `check_data` y el cambio de capa funcionan sin cambios. Sin dependencias nuevas: ni libprotobuf ni zlib.

Pieza	Qué hace
`Gzip` `vector/Gzip.h`	`inflateIfCompressed` : gzip (RFC 1952) o zlib (RFC 1950) con **miniz** (dominio público, el mismo de `fill_hgt`); lo que no está comprimido pasa intacto. Una tesela vacía (gzip de 0 bytes, el mar en OpenMapTiles) es válida.
`mvt::decode` `vector/MvtTile.h`	Lector de protobuf propio (varints, fijos, bloques) → `Tile` / `Layer` (nombre, extent, tablas de claves y valores) / `Feature` (tipo, tags, piezas en coordenadas de tesela). `decodeGeometry` interpreta MoveTo/LineTo/ClosePath en zigzag. Nunca lee fuera del buffer: un dato corrupto da error, no un cuelgue.
`VectorStyle` `vector/VectorStyle.h`	Fondo + reglas en orden (capa, filtro/exclusión por propiedad, zooms, tipo de geometría, relleno, trazo con grosor interpolado por zoom, borde, discontinuo, rótulo, marcador de punto con forma: círculo, cuadrado, triángulo, rombo o cruz). Los dos estilos por defecto traen cinco grupos de puntos de interés por tema (`poi_*`), apagados de partida. `openMapTiles()` trae el estilo por defecto para ese esquema (JSON embebido, = `docs/estilo_vectorial.json`); `fromJson/fromFile` lee uno propio.
`LandMask` `vector/LandMask.h`	Máscara de tierra para pintar el **mar** cuando el MBTiles no trae océano: los polígonos de una BD de entidades (la costa de `osm_costa_a_sqlitedb.py`), en Mercator normalizado con su recuadro. `paint(z, x, y)` rellena la tierra de la tesela (par-impar: las lagunas quedan como agua) y no le pasa a `QPainter` la isla entera: descarta anillos que no tocan, reduce cada racha de puntos que queda al mismo lado fuera de la tesela a sus extremos (no cambia lo que hay dentro) y quita los puntos a menos de 0,3 px.
`featurequery::query` `vector/FeatureQuery.h`	«¿Qué hay aquí?»: la consulta inversa al pintado. Pasa un punto (lat, lon) a coordenadas de la tesela y devuelve las entidades a menos de un radio **en metros** (el tamaño de la tesela depende de la latitud), con sus atributos: distancia 0 si cae dentro de un polígono (los huecos cuentan como fuera), si no al borde, la línea o el punto más cercano. Orden: polígonos que contienen el punto, luego por cercanía. Lo expone `MapWidget::vectorFeaturesAt` (overlay y base vectoriales) y `vector_tile --at` .
`poisearch::inTile / search` `vector/PoiSearch.h`	Búsqueda de puntos de interés en teselas vectoriales, sin índice aparte. `inTile` saca los puntos de la capa `poi` con su posición geográfica (los del margen de la tesela se descartan: los trae la vecina); `search` exige que cada palabra del texto esté en el nombre o en el tipo (español, inglés y palabras de uso común, sin tildes) y ordena: empiezan por lo

	escrito, lo contienen, solo por tipo; luego por cercanía. `mvt::decode` acepta una lista de capas para saltarse el resto sin decodificarlo. `MapWidget::searchPointsOfInterest` lee el rectángulo de teselas z14 alrededor del centro de una sola consulta, y guarda lo leído en una caché. `MapWidget::importPointsOfInterest` es la versión en bloque: lee las teselas de una zona fila a fila (sin caché, con progreso y corte), filtra con `poisearch::filter` (sin tope ni mínimo de letras) y crea una entidad punto por POI en una capa, con el color de su tema (`poisearch::themeColor`) y sin duplicar los que ya estén.
`streetsearch::inTile` / `search` `vector/StreetSearch.h`	Búsqueda de calles en teselas vectoriales, sin índice aparte. `inTile` saca los tramos con nombre (o referencia) de la capa `transportation_name` con su geometría geográfica; `search` filtra por palabras (nombre o referencia, sin tildes; «calle», «avenida»... opcionales), **une los tramos del mismo nombre que se tocan** a menos de 15 m (el solape de las teselas vecinas) en una sola calle y ordena: empiezan por lo escrito, lo contienen; luego por cercanía. `MapWidget::searchStreets` comparte con los POI la lectura de teselas z14 alrededor del centro (`itemsAround` , una consulta por zona y caché) y `goToStreet` encuadra la calle entera.
`roadgraph::Graph` / `routeplanner::findRoute` `vector/RoadGraph.h` · `vector/RoutePlanner.h`	Rutas en coche por la red vial de las teselas (capa `transportation`), sin conexión. `RoutePlanner` lee las teselas z14 del rectángulo origen-destino con margen, fila a fila y solo esa capa, y `Graph` las **cose** (suelda vértices a menos de 1,5 m, 4 m en extremos de vía; un puente no se une con lo de abajo) y busca el camino de menor tiempo con A* (velocidad media por tipo de vía; respeta sentido único y vías cerradas). Origen y destino se enganchan al *punto* más cercano de la vía partiendo el tramo con nodos virtuales. El resultado trae la ruta partida en tramos seguidos de un mismo tipo de vía (`RouteResult::legs`), para pintar cada tipo de un color; el ejemplo `examples/rutas_tierra` lo usa para destacar los caminos de tierra. `routeReach` da el contorno de destinos que caben en el tope de teselas (el ejemplo lo dibuja alrededor de A). `MapWidget::findRoute` , «Ruta (coche)» en la demo y `vector_tile --route` . Sin giros prohibidos ni límites reales: el tiempo es una estimación.
`VectorTileRenderer` `vector/VectorTileRenderer.h`	Grupos ocultos (`setHiddenGroups`): las reglas de un grupo apagado no se pintan. Los nombres de calles y ríos van a lo largo de la línea, letra a letra (centro, un tercio o dos tercios; no si dobla más de 35° o se sale de la tesela). Pinta regla a regla con `QPainter` sobre `QImage` (una ruta por regla, relleno `WindingFill` para respetar huecos). Los rótulos van al final, sin solaparse, solo para puntos **dentro** de la tesela y desplazados para caber enteros: nada sale cortado ni repetido entre teselas. Reentrante; sin `QGuiApplication` omite los rótulos.

Cómo se engancha. `MBTiles::describe` decodifica una tesela real: si es MVT pone `TileDataset::vector` (un fichero que dice `pbf` pero no lo es se rechaza con su motivo). `TileDataset::vectorStyleFile` (`"vectorStyle"` en `datasets.json` / `mapa.json` , relativo al JSON y parte de `DataPackage::files()`) elige el estilo; si falta o no se lee, el de OpenMapTiles. `TileDataset::vectorLandMask` / `vectorLandLayer` (`"landMask"` / `"landLayer"`) dan la máscara de tierra: con ella el fondo de la tesela es `VectorStyle::sea` y la tierra se pinta encima con

`background`, antes de las reglas. `TileLoader::rendererFor` crea un renderizador por capa y `renderVector` reparte las teselas de una rejilla entre su hilo y un `QThreadPool` propio (núcleos − 2), abandonando si la petición caduca: una tesela z14 urbana (~8600 edificios) cuesta 50-130 ms, mucho más que un PNG. Con `vector = false` (todo lo anterior) el camino es el de siempre, línea a línea.

Grupos y mapa híbrido. Cada regla del estilo puede pertenecer a un **grupo** («Nombres de calles», «Puntos de interés», «Edificios»…; `VectorRule::group`, títulos en `"groups"`, y `"visible": false` para los que arrancan apagados). `TileDataset::vectorHiddenGroups` (`"hiddenGroups"`) da el estado de partida y `TileService::setVectorHiddenGroups` lo cambia en caliente. Para eso pone en el `TileLoader` un renderizador nuevo (el viejo puede estar pintando en el pool), vacía la caché de la capa y tira lo que llegue de peticiones anteriores (`m_discardBefore`). El **mapa híbrido** es el mismo MBTiles en un **segundo `TileService`** (hilo y caché propios) con `TileDataset::vectorOverlay`, que elige el estilo híbrido (`VectorStyle::hybrid`: fondo transparente, vías finas, rótulos blancos con halo). Su nivel de fondo es el último, para no ampliar mucho texto mientras carga. `MapView::setOverlayService` lo dibuja con otro `TileLayer` transparente, en la capa `tileoverlay` (encima de `hillshade`, debajo de las entidades). `requestVisibleTiles` / `refreshPlan` piden y planifican las dos. API: `MapWidget::setVectorOverlay`, `vectorOverlayGroups`, `setVectorOverlayGroupVisible`, `setVectorOverlayVisible` y `baseLayerGroups` / `setBaseLayerGroupVisible` para una base vectorial; en `mapa.json`, `"vectorOverlay"`.

7 · Módulo `widget` — el mapa en pantalla

Convierte el motor de teselas en un `QWidget` embebible. El motor de dibujo es QCustomPlot, pero queda oculto.

`MapWidget` include/libmapa/MapWidget.h · QWidget (PIMPL)

La **cara pública** de toda la librería. No expone QCustomPlot: por dentro tiene un `Impl` con el `TileService` y el `MapView`. Agrupa toda la API que usa la aplicación:

Grupo	Funciones (ejemplos)
Capa base	`availableBaseLayers` · `baseLayerId` · `setBaseLayerId` · `reloadBaseLayer`
Navegación	`center/setCenter` · `zoom/setZoom` · `zoomIn/Out` · `fitBounds` · `visibleNorthWest/SouthEast` · **`setMaxZoomOverride`**
Capas de entidades	`addFeatureLayer` · `setFeatureLayerVisible` · `setFeatureLayerZOrder`
Entidades	`addFeature` · `updateFeature` · `removeFeature` · `features` · `loadGeoAsLayer` · `loadFeaturesAsLayer` · **`importVectorFile`** · **`exportVectorFile`** · `setFeatureLayerZoomRange` · `setFeatureLayerTransient`
Edición geometría	`moveVertex` · `insertVertex` · `removeVertex` · `moveFeature` · `undo/redo`
Paquete de datos	`MapConfig::dataDir` · `packageInfo` · `dataWarnings`
Persistencia	`saveFeaturesTo` · `loadFeaturesFrom` · `setFeaturesDbFile` · `saveFeaturesNow`
Objetivos móviles	`addTarget` · `updateTarget (con hora)` · `targetHistory` · `targetOrigin` · `setTargetHistoryLimit` · `setTargetHistoryMinDistance` · `setSelectedTargetHistoryVisible` · `setTargetLabel` · `setTargetAttribute` · `targetAttribute` · `setTargetSymbolProvider` · `setTargetDetailBudget` · `setTargetTrailLength` · `targetAt` · `setSelectedTarget` · **`clearSelectedTarget`** · **`targetSelectionChanged`**
Alertas de zona	`watchZone` · `unwatchZone` · `isZoneWatched` · `watchedZones` · `targetsInZone` · `zonesOfTarget` · `featuresInZone` · `zonesOfFeature` · `setZoneAlertHighlight` · `setZoneAlertColor` · **señales `targetEnteredZone` / `targetLeftZone` / `featureEnteredZone` / `featureLeftZone`** · consulta puntual **`targetsInArea`** · **`featuresInArea`**
Agrupación de objetivos	**`setTargetClustering(TargetClustering)`** · **`targetClustering`** · **`targetClusterAt`** · **señal `targetClusterClicked`** · núcleo: `MapDecor::clusterPoints` (rejilla anclada al mundo)
Filtro de objetivos	**`setTargetFilter(TargetFilter)`** · **`clearTargetFilter`** · **`hasTargetFilter`** · **`targetPassesFilter`** · **`filteredOutTargets`** (clase, velocidad, zonas vigiladas, predicado de la app; ocultar o atenuar)
Anillos y marcaciones	**`hitTest`** · **`contextMenuRequested (MapHit)`** · **`addRangeRings`** · **`updateRangeRings`** · **`rangeRings`** · **`rangeRingsSpacing`** · **`addBearingLine`** · **`updateBearingLine`** · **`bearingLine`** · **`removeRangeMark`** · **`clearRangeMarks`** · **`rangeMarks`** · **`bearingLineMeasure`** · inicio/fin de rutas: **`setRouteEndpoints (Hidden/Selected/Routes/AllLines)`**

Rótulos	**featureLabelRect · targetLabelRect · setDefaultTargetLabelOffset · defaultTargetLabelOffset · clearTargetLabelOffset · setTargetLabelFormat (TargetLabelFormat) · setTargetLabelSelectionColor · targetLabelMoved · setTargetLabelDragButton · setTargetLabelSelected · clearTargetLabelSelected · targetLabelSelectionChanged · targetLabelClicked**
Reproducción	**TrackPlayer** (loadFile · setTracks · play · pause · stop · seek · setSpeed · setRoutesVisible · targetIdOf · isLoaded · **clear = salir** · señales timeChanged / playingChanged / finished / cleared) · núcleo: **sampleTrack · trackTimeRange** (TrackReplay.h)
Proximidad (CPA/TCPA)	**setProximityAlert(distancia, horizonte) · proximityAlerts · targetCpa · setTargetVelocity · setProximityLinesVisible · setProximityColor · señales proximityAlertRaised / proximityAlertCleared**
Exportar	**renderImage(escala) · exportView(ruta, MapExportOptions)** (PDF vectorial con título, pie y escala 1:N, o PNG/JPG…) · **exportTargetHistory** (GPX con horas)
Herramientas / ratón	setActiveTool (Measure, **MeasureArea** → areaMeasured, AreaZoom, SelectArea, SelectPolygon, Draw…, EditFeature) · clearAreaSelection (quita el recuadro o el contorno de la selección)
Elevación	elevationAt · elevationProfile · lineOfSight · viewshed
Escala / cuadrícula	setScaleBarVisible · setScaleBarUnits (Metric/Nautical) · metersPerPixel · setGraticuleVisible · setGraticuleColor · **setGraticuleMode (Geographic/Utm)**
Cobertura / relieve	setCoverageVisible · setCoverageZoom · refreshCoverage · **setHillshadeVisible · setHillshadeSun · setHillshadeOpacity · setHillshadeExaggeration · setHillshadeColored · refreshHillshade**
Conversión	toAxisCoords · fromAxisCoords
Señales	zoomChanged · centerChanged · areaSelected · clicked · featureCreated…

Exportar la vista. `exportView` elige el formato por la extensión. En **PDF** (`QPdfWriter` , de QtGui) dibuja el mapa con `QCustomPlot::toPainter` sobre un `QCPPainter` vectorial, a la resolución lógica de la pantalla: la hoja es una copia fiel de la vista, con entidades, objetivos, escala y cuadrícula como vectores y las teselas a su resolución nativa. La hoja (A4 por defecto) se orienta sola, y el pie da la extensión, la fecha UTC y la escala de impresión 1:N calculada en el centro. En **imagen**, `renderImage(escala)` redibuja a más píxeles y el título y el pie van en bandas blancas. Las dos se escriben de forma atómica (`QSaveFile`).

`MapView` widget/MapView.h · QCustomPlot

El lienzo real: hereda de QCustomPlot, gestiona el ratón (arrastrar, rueda, herramientas), traduce píxeles↔coordenadas y coordina las capas. Sin herramienta, un clic sobre la **caja de una etiqueta** (de entidad u objetivo) la **arrastra** —cambiando su desplazamiento relativo— con prioridad sobre seleccionar o desplazar. Un **punto marcado** `draggable` (p. ej. un pin A/B) se arrastra igual, moviéndose en el modelo y avisando con las señales `featureDrag*` . El trazado punto a punto admite un **tope de vértices** (`setDraftMaxVertices` , que persiste como atributo `max_vertices`): un «área objetivo» de ≤10 puntos es un polígono con `type="objetivo_area"` y la etiqueta anclada a su

primer vértice (`labelAnchorVertex=0`), sin un tipo de geometría nuevo. `requestVisibleTiles()` pide teselas del viewport; `refreshPlan()` rehace el plan y repinta. Aquí vive el rectángulo de `SelectArea` que se añadió para la descarga.

Capas de dibujo (QCPLayerable)

Clase	Rol
`TileLayer`	Pinta las teselas del plan (con la escalera de respaldo). Lee del `TileCache` .
`FeatureLayer`	Pinta las entidades vectoriales de una capa (según `OverlayModel`) y sus tiradores al editar. **Recorte por viewport**: salta (sin proyectar a píxeles) las entidades cuya caja no toca la vista, y respeta el **rango de zoom** de la capa — así una capa densa (curvas de nivel) no frena el *pan*. Un punto con icono se ancla por `FeatureStyle::iconAnchor` (normalizado; `(0.5,1.0)` = la punta del pin marca el lugar) y su **hit-test** cubre todo el pixmap, para poder agarrarlo.
`TargetLayer`	Pinta los objetivos móviles y su traza. Se repinta sola en modo BUFFERED sin rehacer teselas. El símbolo lo decide el proveedor de la app (icono + rotación por rumbo + escala); sin proveedor, un galón por defecto. Para miles de objetivos: culling por vista, nivel de detalle (presupuesto de etiquetas/trazas), declutter de etiquetas y decimación de traza. El **rótulo** de un objetivo se coloca con el offset efectivo (`MapTarget::labelOffsetPx` si lo fijó la app o el operador, `labelOffsetSet` ; si no, el de `MapWidget::setDefaultTargetLabelOffset` , `(0,0)` de partida), admite **colores por segmento** (`MapTarget::labelContent`) y su aspecto común lo da `TargetLabelFormat` : colocación `Classic` (la de siempre) u `Hanging` (la caja cuelga de `posición + offset` por la esquina más cercana al objetivo, `labelCornerFor` , en `LabelPlacement.h`), fuente monoespaciada, ancho en caracteres y columnas por segmento. Se arrastra con el ratón (`MapView`), con el botón que elija la app (`setTargetLabelDragButton` : izquierdo por defecto o derecho, y entonces el clic derecho sobre el formulario no abre el menú contextual): `targetLabelMoved` avisa al soltar y soltar fuera del mapa cancela. Agarrar el formulario y seleccionar el objetivo son **dos estados distintos**: el objetivo resaltado (`m_selected` : halo y rótulo forzado) y el formulario agarrado (`m_labelSelected` : un recuadro del color de selección mientras dura el arrastre, sin halo).
`LabelLayer`	Capa de **rótulos**: pinta encima de todo el texto que acompaña al mapa. Las entidades, anillos y marcaciones se dibujan en su capa y los objetivos encima; si cada uno pintara su texto, una marcación o un anillo taparían el nombre de una zona. Por eso `FeatureLayer::drawLabels` (nombre de entidades y alertas CPA) y `RangeLayer::drawLabels` (distancias de anillos y rótulo de marcación, con línea guía si está desplazado) se pintan aquí: misma capa *buffered* «targets», creada la última. No guarda estado.
`GraticuleLayer`	Cuadrícula de meridianos y paralelos con rótulos en el borde, o **UTM** (líneas de este/norte por zona, 100 km–100 m, curvas muestreadas y recortadas a la zona, límites de zona marcados). Paso automático según el zoom (pasos de carta), paralelos en la proyección del mapa. Encima de las entidades y debajo de los objetivos.
`ScaleBarLayer`	Escala gráfica abajo a la izquierda, medida en el centro de la vista con la distancia geodésica real entre dos píxeles (en Mercator la escala cambia con la latitud). Encima de todo.
`CoverageLayer`	Mancha de **cobertura**: qué zonas de un zoom objetivo ya están en la BD, agregadas a una rejilla gruesa y coloreadas por completitud (ámbar→verde). Fija: se ve aunque mires a otro zoom.

`HillshadeLayer`	**Relieve sombreado** (hillshade) en vivo: capa «tonta» que pinta una imagen —calculada por `MapWidget` del DEM LOCAL (sin conexión)— estirada entre los píxeles de sus esquinas (alineada con la base, eje X=longitud, Y=grados de Mercator). Va encima de las teselas y debajo de las entidades. Gris en *Multiply* sobre la base, o tintado por altura (hipsométrico) en *SourceOver*.

Modelos (QObject, sin dibujo)

Clase	Rol
`OverlayModel`	Guarda las entidades y capas, la selección y el historial de **deshacer/rehacer**. Separado del dibujo para poder probarlo sin ventanas.
`TargetModel`	Guarda los objetivos, su traza dibujada (acotada) y su **historial completo** con hora desde el **origen** (donde se vio por primera vez), compacto (24 B/punto), con distancia mínima y aclarado al tope que nunca pierde origen ni último punto. Operaciones O(1) por objetivo para el tiempo real; emite `changed()`.

8 · Módulo `io` — ficheros .geo y paquete de datos

`GeoFile` include/libmapa/GeoFile.h lee/escribe ficheros `.geo` / `.xyz`: listas de coordenadas donde `0,0` separa trazados. Un fichero puede traer varios trazados (aguas jurisdiccionales, corredores, divisiones). `MapWidget::loadGeoAsLayer()` lo carga como **una entidad multi-parte**. También lo usa `geo_to_tiles` para rasterizar.

`VectorFormats` include/libmapa/VectorFormats.h

Intercambio con otras herramientas (QGIS, Google Earth, GPS): lee y escribe `MapFeature` en **GeoJSON**, **KML** y **GPX**, eligiendo el formato por la extensión. Escritura atómica (`QSaveFile`). Sin dependencias nuevas: JSON y XML de Qt Core. Probado sin ventanas en `tst_vectorformats`.

Formato	Qué conserva
GeoJSON	Todo: geometría (Point/LineString/Polygon y sus Multi*), nombre, descripción, tipo de dominio (`libmapa_type`), atributos y estilo «simplestyle» (`stroke`, `stroke-width`, `fill` …).
KML	Geometría (MultiGeometry = partes), nombre, descripción, tipo y atributos en `ExtendedData`, estilo en línea o por `styleUrl` (colores `aabbggrr`). Carpetas aplanadas; KMZ no.
GPX	Puntos (`wpt`, con `ele`), rutas y tracks (un `trkseg` por parte), nombre, descripción y tipo. Sin atributos ni estilo; los polígonos van como track cerrado con la extensión `libmapa:polygon` y vuelven a ser polígonos al releerlos.

Los huecos de los polígonos se ignoran (el modelo no los tiene). `MapWidget::importVectorFile` añade las entidades de un fichero a una capa; `exportVectorFile` escribe todas o las de una capa.

Recorridos con hora. `GpxTrack` (nombre, tipo, id de objetivo y puntos `TargetTrackPoint`) con `tracksToGpx` / `writeGpxTracks` / `parseGpxTracks`: un `<trk>` por recorrido con `<time>` UTC (ISO 8601 con milisegundos) en cada punto y el id como extensión `libmapa:targetId`. Es lo que usa `MapWidget::exportTargetHistory` para sacar el historial de los objetivos; QGIS o GPXSee lo reproducen en el tiempo.

`DataPackage` io/DataPackage.h · público: DataPackageInfo

El producto final trabaja **sin conexión**: todos sus datos viajan en un **paquete**, una carpeta con un manifiesto `mapa.json` (formato `libmapa-package`, versión 2) y rutas **relativas** a ella. `DataPackage::load()` lo lee (carpeta o fichero), valida formato y versión, resuelve las rutas a absolutas y anota como **aviso**, sin fallar, lo que falte. Un `datasets.json` de la versión 1 se acepta como paquete mínimo.

`package`	id, nombre, versión de los datos, fecha, zona (`bounds`) y atribución → `MapWidget::packageInfo()`.
`start`	Capa, centro y zoom de arranque.
`datasets`	Capas base (mismo formato que `datasets.json`; lo omitido toma su valor por defecto).
`elevation`	`file` (BD `.sqlitedb`) o `dir` (carpeta `.hgt`).
`overlays`	Capas vectoriales **fijas** (bloqueadas, excluidas del guardado de entidades, `fixedLayers`): un `.geo` con `style` (`loadGeoAsLayer`), o una BD `.sqlitedb` de entidades (`loadFeaturesAsLayer`) que conserva el **estilo/etiqueta/atributos por entidad** — así una capa de

	curvas de nivel de `dem_to_contours` entra como overlay. Son **decoración de fondo**: arrancan **ocultas y no seleccionables** (`LayerInfo::visible` / `LayerInfo::selectable` en falso; `"visible"` y `"selectable"` en el overlay del `mapa.json` las activan, y `setFeatureLayerSelectable` lo cambia en caliente), porque un polígono grande (una provincia) se llevaba el clic de lo que hay encima. `featureAt` las sigue viendo (un tooltip o una lectura de cota no seleccionan nada); `hitTest` y el ratón no. `minZoom` / `maxZoom` (por capa, en `LayerInfo`) limitan el zoom en que se dibuja y se selecciona: una capa de detalle (curvas) se oculta a vista general para no ensuciarla ni frenarla.
`features`	BD de entidades del usuario. Relativa → `AppDataLocation/<package.id>/` (escribible), nunca la carpeta del paquete; `seed` se copia la primera vez.

`MapConfig::dataDir` es el único ajuste necesario; cualquier campo de `MapConfig` rellenado a mano tiene prioridad sobre el paquete.

`PackageCheck` io/PackageCheck.h

Comprueba que un paquete funcionará **sin conexión**: lo que falte no se podrá descargar después. Cada hallazgo lleva su gravedad: **Error** si una parte del mapa no se dibujará, **Warning** si funciona pero algo falta o no es portable, **Info** para el informe. No escribe nada (todo en solo lectura; la BD de entidades del usuario ni se abre, porque abrirla podría migrar su esquema).

`Qt`	Driver `QSQLITE` disponible (sin él no abre nada).
`Capas base`	El fichero existe, abre como base de teselas, y una tesela real del nivel de fondo **se decodifica**: detecta el plugin de imagen que falta al desplegar (`qjpeg`) y lo nombra.
`Cobertura`	Solo en modo completo: por cada zoom, teselas presentes / esperadas dentro de `bounds` , con `RMapsTileSource::countInRange` (un `COUNT(*)` que usa el índice: ~0,2 s para 7 GB).
`Elevación / capas fijas`	La BD tiene `dem_tiles` con datos; los `.geo` se leen.
`Paquete`	Atribución y zona declaradas, capa de arranque existente, ids no repetidos, y todo **dentro** de la carpeta (lo de fuera no viaja).

El modo **rápido** (sin cobertura) lo ejecuta `MapWidget` al abrir un paquete y deja los problemas en `dataWarnings()` ; el mapa arranca igual con lo que funcione. El completo lo usa la herramienta `check_data` .

8b · Módulo `dem` — elevación del terreno (HGT/SRTM y GeoTIFF)

Da la cota en metros de una coordenada, leyendo datos **SRTM**. Vive en `libmapa_core` (sin widgets): se consulta igual desde la ventana que desde consola o un test. Separa *de dónde salen las muestras* de *cómo se interpola*, con una interfaz al estilo de `ITileSource`:

- **`IElevationSource`** src/dem/IElevationSource.h: interfaz pura, un método `elevationAt(coord)→double` (contrato NaN).
- **`GridElevation`** src/dem/GridElevation.h: base que concentra lo común (caché LRU de tiles, lectura de muestra, **interpolación bilineal**, huecos→NaN); deja un solo hueco: `loadTile()`.
- **`HgtElevation`**: `loadTile()` lee el fichero `.hgt` de una carpeta.
- **`SqliteElevation`**: `loadTile()` consulta una base de datos `.sqlitedb` y descomprime el blob del tile — un **solo fichero**, cómodo para empaquetar en una app.
- **`GeoTiffElevation`** src/dem/GeoTiffElevation.h: lee uno o varios **GeoTIFF** (fichero o carpeta). No hereda de `GridElevation` (no son tiles de 1°): implementa `IElevationSource` directamente, con un índice por celdas de 1° para elegir el fichero y su propia interpolación bilineal.
- **`GeoTiffImage`** src/dem/GeoTiff.h: el lector de TIFF, **sin GDAL**: TIFF clásico y BigTIFF, franjas o teselas, sin compresión / LZW / Deflate / PackBits con predictores 2 y 3, enteros y coma flotante, georreferencia en grados o Web Mercator y valor «sin dato». `verifyBlocks()` revisa todos los bloques (descarga cortada, bloques ilegibles; herramienta `geotiff_check`). Descodifica solo los bloques que se consultan, en una caché **única y compartida** por todas las imágenes (256 MiB por defecto, `setCacheLimit`): una carpeta con decenas de cuadros no multiplica el consumo. Seguro entre hilos.
- **`openElevationSource(ruta)`** src/dem/ElevationSources.h: elige la fuente por la ruta — `.tif` o carpeta con `.tif` y sin `.hgt` → GeoTIFF; carpeta con `.hgt` → `HgtElevation`; cualquier otro fichero → `SqliteElevation` (como siempre). Lo usan `MapWidget` (configuración y `setElevationDb/Dir`) y `PackageCheck`.

El `.hgt` es crudo y **sin cabecera**: una rejilla **cuadrada** de muestras `int16` **big-endian** que cubre 1°×1°. El nombre marca la esquina suroeste (`N19W077.hgt` = 19–20 N, 77–76 O); la fila 0 es el borde **norte** y la columna 0 el **oeste**. La resolución no está escrita: se deduce del tamaño (`lado = isqrt(bytes/2)` → 1201 = 90 m, 3601 = 30 m).

Función	Qué hace
`elevationAt(coord) → double`	Localiza el tile (floor de lat/lon), autodetecta el lado, lee big-endian e **interpola bilinealmente**. **NaN** si no hay dato (tile ausente, sin origen, o hueco SRTM `-32768`).
`loadTile(lat,lon,…)`	El único método que cambia por origen: lee el fichero (`HgtElevation`) o el blob de la BD (`SqliteElevation`).

El `.hgt` es crudo y **sin cabecera**: rejilla **cuadrada** de `int16` **big-endian** de 1°×1°, nombre = esquina SO (`N19W077.hgt`), fila 0 = norte, col 0 = oeste. La resolución se deduce del tamaño (`lado = isqrt(bytes/2)` → 1201 = 90 m, 3601 = 30 m). La **BD** guarda esas mismas muestras `qCompress`-adas: `dem_tiles(lat,lon,side,data)` + `dem_meta`; se abre read-only por el `SqliteConnectionPool`.

Propiedades: caché LRU de tiles abiertos; huecos→NaN no inventa terreno; dos orígenes carpeta `.hgt` o BD empaquetable, misma cota. Se expone por la fachada: `MapConfig.elevationDir` / `elevationDbFile`, `MapWidget::elevationAt()` y `setElevationDir/Db()`; `fill_map` muestra la cota bajo el cursor (`--dem` carpeta, `--dem-db` base de datos, o el botón **DEM…**).

8c · Análisis de elevación — perfiles sobre el DEM

Sobre ese mismo `IElevationSource`, el núcleo calcula **perfiles del terreno** con funciones libres en `src/dem/ElevationAnalysis.{h,cpp}` y tipos de resultado públicos en `include/libmapa/Elevation.h`. Son cálculos de **visibilidad directa** (nada de radar). La primera capacidad es el **perfil de una ruta**.

Elemento	Qué hace
`elevationProfile(src, path, params)`	Cota del terreno a lo largo de una polilínea, muestreada a paso constante (por defecto **30 m**) de forma uniforme por toda la ruta (incluye siempre el último punto exacto). Camina rayos geodésicos con `atDistanceAndAzimuth + GeoMath`.
`ElevationProfile`	`samples[]` (`{distanceM, position, elevation}`, NaN = hueco), `totalDistanceM`, `min/maxElevation`, `gain`, `loss`. Las estadísticas ignoran los NaN.
`lineOfSight(src, a, b, antA, antB, params)`	Visibilidad directa entre dos puntos con altura de antena en cada extremo. Muestrea el terreno intermedio y lo compara con la recta entre las cimas de antena, **más el abombamiento de la Tierra** (radio efectivo **k·R**, k=4/3; `curvature=false` → geométrico).
`LineOfSightResult`	`clear` (¿hay visión?), `clearanceM` (holgura mínima; negativa = cuánto falta), `blockPosition` / `blockDistanceM` (punto crítico), `isValid()` (falso sin dato en un extremo).
`computeViewshed(src, origin, params, progress)`	Viewshed 360°: un rayo por azimut (`azimuthStepDeg`) hasta `maxRangeM`. Por rayo: ángulo de cierre del terreno, horizonte acumulado y **picos** de la silueta, y la visibilidad de un objetivo a `targetHeight`. Curvatura k·R como en `lineOfSight` (y **consistente punto a punto** con él). `progress` (opcional, `ViewshedProgress = function<bool(hecho,total)>`) se invoca tras cada azimut; devolver `false` cancela (resultado vacío) → barras de progreso en alcances largos (hasta 400 km).
`Viewshed / ViewshedRay / VisibleRange`	`Viewshed{origin, rays[]}`; cada `ViewshedRay{azimuthDeg, horizonDeg, peaks[], visibleRanges[], hiddenRanges[], visibilityReachM, profile}`. `visibleRanges` = TODOS los tramos visibles del rayo (la **zona real, con huecos**); `hiddenRanges` = los ocultos (ambos parten el rayo cubierto, para sombrear); `visibilityReachM` = solo el primer tramo contiguo. `keepProfiles` llena `profile` por rayo.
`voidElevation (en los 3 params)`	Cota mínima del análisis (**suelo**). NaN (por defecto) = cota cruda (salta los huecos; mantiene los negativos de tierra firme). `0` = mar: sustituye los huecos por 0 **y sube a 0 la batimetría negativa** (el fondo marino), porque sobre el mar la superficie que cuenta para la visibilidad es el nivel del agua, no el lecho — ni un objetivo flota en el fondo ni el fondo tapa una visual de superficie. Así los objetivos sobre el mar se analizan hasta el alcance en vez de cortar en la costa.
`computeContours(src, params)`	Curvas de nivel (isohipsas) por **marching squares** sobre una rejilla muestreada del DEM (`include/libmapa/Contours.h`). Por celda y nivel (cota = `base + k·interval`) emite segmentos y los **encadena** en polilíneas uniendo por arista de rejilla (cada cruce tiene clave exacta, sin tolerancias). Streaming por filas (solo 2 filas de nodos en memoria). `ContourParams` acota con `minLevel` / `maxLevel` (p. ej. `minLevel=0` recorta al nivel del mar y omite la batimetría). `progress` opcional (`ContourProgress = function<bool(hecho,total)>`) se invoca por fila y permite cancelar → barra

	de progreso al hornear país entero. Devuelve `ContourLine{elevation, points[]}` ; celdas con algún vértice sin dato se saltan. La herramienta `dem_to_contours` las guarda como capa vectorial (índice cada N m, con etiqueta de cota).

El **perfil de ruta** es altura **real** del terreno EN EL LUGAR —no aplica curvatura terrestre—, para dibujar el corte del recorrido o medir desniveles. La **visibilidad punto a punto** y el **viewshed 360°** sí corrigen la curvatura+refracción (la constante `d1·d2/(2·k·R)` es la misma física que `DVD_potencial`, k=4/3; la caída desde el observador es `d²/(2·k·R)`); son visibilidad **directa**, no radar: el horizonte geométrico (`≈4.12·√h`) **emerge** de la geometría (atan2 + caída), no se codifica. La visibilidad del viewshed por muestra es, término a término, la misma condición que `lineOfSight` (la ZVD y la Visión A→B no se contradicen en un azimut). Un viewshed de 360° a 30 m y 50 km tarda ~0,1 s. La librería devuelve los datos; la gráfica la pinta la app. Se exponen por la fachada: `MapWidget::elevationProfile(path, params)`, `MapWidget::lineOfSight(a, b, antA, antB, params)` y `MapWidget::viewshed(origin, params)` (vacío / inválido si no hay origen DEM configurado). La app `demo` lo prueba en su **pestaña «Elevación»** del panel lateral: el perfil se abre en una ventana flotante interactiva (ejes/rejilla, zoom con la rueda, arrastre, cursor con lectura); la Visión A→B se dibuja sobre el mapa y sobre el perfil (azul visible, rojo oculto, obstáculo con su distancia); y el viewshed sombrea **azul lo visible y amarillo lo oculto**, con opción de tratar el mar/sin dato como cota 0.

8d · `libtelemetria` — decodificador de NMEA 0183 y AIS

Una **biblioteca aparte** (carpeta `libtelemetria/`), no un módulo de libmapa: convertir líneas de texto en datos no tiene nada que ver con teselas ni ventanas, y así la usa cualquier app sin arrastrar el mapa. Depende **solo de Qt Core** (el `CMakeLists.txt` se detiene si alguien le añade Network, Gui, Sql, Positioning o libmapa) y no hace E/S: se le dan bytes y devuelve datos. Namespace `telemetria`; se enlaza con `telemetria::telemetria` y se instala en el **mismo prefijo** que libmapa (`find_package(libtelemetria)`; con qmake, `telemetria.pri` junto a `libmapa.pri`).

- **`Sentence` / `parseSentence`** include/telemetria/Nmea.h: separa una línea en dirección, emisor, tipo y campos (los vacíos cuentan) y comprueba la **suma de comprobación** (XOR). Dice por qué rechaza una línea y cuál debería ser la suma. `formatSentence` hace lo inverso.
- **`Rmc`, `Gga`, `Vtg`, `Hdt`** + `decodeXxx` / `formatXxx`: posición, velocidad y rumbo propios. Campos ausentes = NaN (no se inventa nada); estado V de la RMC = sin fijado; la coordenada se redondea con enteros para no escribir «60.0000» minutos.
- **`AisMessage` / `decodeAis` / `encodeAis`** include/telemetria/Ais.h: los 27 tipos de ITU-R M.1371: posiciones (1, 2, 3, 18, 19, 27), estáticos (5 en dos frases, 24 partes A y B), estación base y hora UTC (4, 11), aeronave SAR (9), ayuda a la navegación (21), avisos de seguridad (12, 14), acuses y peticiones (7, 10, 13, 15, 16), correcciones DGNSS (17), enlace y canales (20, 22, 23) y binarios (6, 8, 25, 26) con la carga de aplicación SIN interpretar. «No disponible» (181°, 91°, 1023, 3600, 511) se decodifica como NaN/−1, no como dato. Los tipos reservados (0, 28 a 31) devuelven solo tipo y MMSI. `TELEMETRIA_API_LEVEL` es 3.
- **`AisAssembler`**: reensambla los mensajes de varias frases por (secuencia, canal), con memoria acotada (32 pendientes) y contando los fragmentos perdidos.
- **`Decoder`** include/telemetria/Decoder.h: lo que usa una app. `feed` para un flujo (TCP, fichero: la línea a medias espera), `feedDatagram` para UDP (cada datagrama trae líneas completas). Devuelve `Event`s (Rmc, Gga, Vtg, Hdt, Ais con `ownShip` para !AIVDO, Bad, Ignored) y lleva `DecoderStats`: líneas, decodificadas, suma mala, mal formadas, ignoradas, AIS completos/no soportados, fragmentos perdidos.

Verificación: 3 tests (`tst_nmea`, `tst_ais`, `tst_decoder`). Las sumas publicadas de RMC (6A), GGA (47) y VTG (48) y el mensaje AIS de la documentación de gpsd (MMSI 477553000, 47.582833° N, 122.345833° O, suma 5C) salen exactos; el resto, ida y vuelta de todos los tipos. Apps de ejemplo: `emisor_nmea` (simula y codifica) y `cliente_telemetria` (decodifica y dibuja con libmapa).

9 · Flujos de interacción entre objetos

9.1 · Pintar el mapa mientras navegas

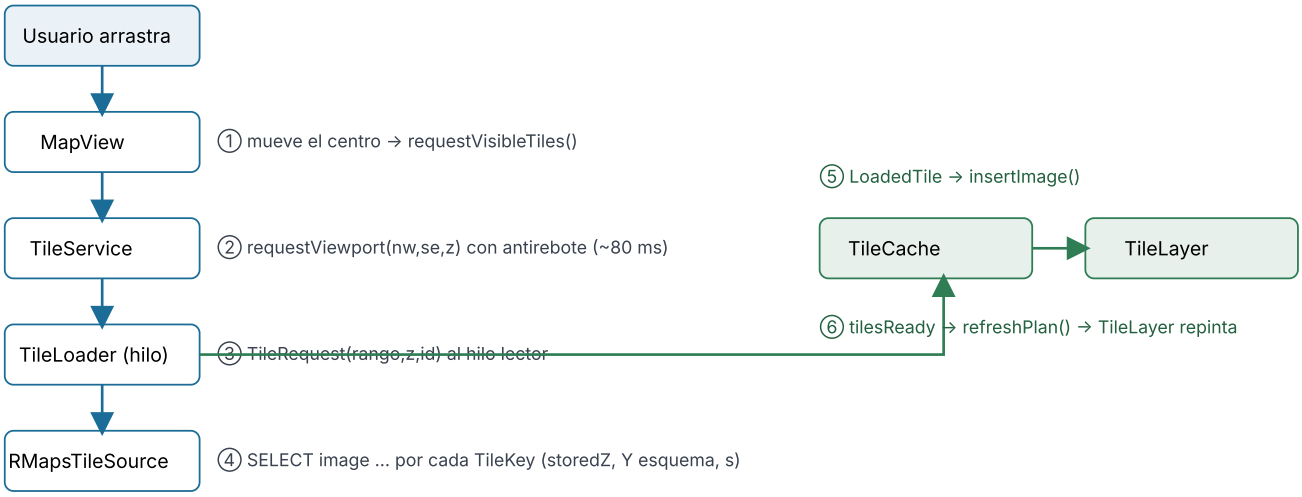


Figura 2 — De un arrastre a la imagen en pantalla. El hilo lector nunca bloquea la interfaz; el caché evita releer.

9.2 · Objetivos móviles en tiempo real

`MapWidget::updateTarget(id, pos, rumbo)` → `TargetModel` actualiza posición y traza en O(1) y emite `changed()` → `TargetLayer` **agrupa** los avisos y repinta como mucho ~30 veces/segundo, y **solo** su capa (modo BUFFERED), sin tocar teselas ni entidades. Con culling por vista, nivel de detalle y declutter escala a **miles** de móviles. Cada objetivo es agnóstico del dominio (`kind` + `attributes` libres) y la app pone su simbología por un hook; la app de ejemplo `demo` lo demuestra moviendo buques, aeronaves y UAVs con su icono orientado al rumbo. Un clic sobre un objetivo (sin herramienta) lo resalta y emite `targetClicked(id, pos)` (`targetAt(pixel)` hace el hit-test); la app lee entonces sus `attributes` para mostrarlos. Cada objetivo guarda además su **historial completo** (posición y hora) desde que se vio por primera vez; al seleccionarlo se dibuja entero, del **origen** (marcado con su hora) a la posición actual, aunque la traza que se dibuja para todos sea corta.

Alertas de zona. `watchZone(idPoligono)` convierte una entidad poligonal en zona vigilada: cada `addTarget` / `updateTarget` pasa por `ZoneMonitor` y, si el objetivo cruza el borde, se emite `targetEnteredZone(objetivo, zona)` o `targetLeftZone`. Las ediciones de la zona (vértices, arrastre, deshacer, cargas en bloque) la reevalúan; borrar el objetivo o la zona no genera salidas. También se vigilan las **entidades punto** (marcadores que el operador coloca o arrastra) con sus propias señales, y la zona ocupada se **resalta en el mapa** (borde y velo rojos, aro en los puntos). Qué significa la zona («prohibida», «de interés»...) lo decide la app por su `type`: la librería solo hace la geometría.

Anillos de distancia y marcaciones. `RangeLayer` es otro `QCPLayerable` de la capa *buffered* «targets», creado antes que `TargetLayer`. Así los anillos (`RangeRings`) y las marcaciones (`BearingLine`) que cuelgan de un objetivo se repintan con él, sin tocar teselas ni entidades, y los objetivos quedan encima. Sus textos no los pinta él: los anota y los pinta `LabelLayer`, por encima de todo; el rótulo de una marcación se arrastra con el ratón (`BearingLine::labelOffsetPx`, `MapWidget::setBearingLabelOffset` / `bearingLabelAt`) y queda unido a su línea por una línea guía. Los anillos son **geodésicos** (`MapDecor::geodesicCircle`): en Mercator se estiran con la latitud, como la distancia real. Las marcaciones siguen la ortodrómica y miden rumbo y distancia al dibujar

(«202° · 26.7 NM», en las unidades de la escala). Una marcación sin longitud (mismo objetivo o mismo punto en los dos extremos) se rechaza. El centro de los anillos puede ser un objetivo, una **entidad punto** (`featureId` : un radar u observador colocado por el usuario; las marcaciones también admiten una entidad punto en cada extremo, `fromFeature` y `toFeature` ; los anillos la siguen y su símbolo queda a la vista, sin marca de centro) o un **punto fijo con nombre**. Con `count` (hasta 100) cuenta solo anillos: los radiales (`spokeEveryDeg`) no suman, y `majorRingEvery` , `majorSpokeEveryDeg` , `majorColor` y `spokeColor` dan énfasis a uno de cada N, para una escala polar de 300 o 600 km en un solo objeto. Con `spacingMeters = 0` la separación es **automática**: en cada repintado se mide la escala local en el centro y `MapDecor::niceRingSpacing` elige el mayor valor 1-2-2.5-5 × 10^n (en la unidad de la escala) con el que los anillos caben en el 45 % del lado corto de la vista. Si la parte de arriba de un anillo, o el extremo de un radial, queda fuera de la vista, su rótulo se lleva al punto visible más cercano.

Clic derecho. `MapView` emite el clic derecho (salvo mientras se dibuja, que lo cierra) y `MapWidget` lo convierte en `contextMenuRequested(MapHit, globalPos)` : `MapHit` trae la coordenada y, por separado, el objetivo suelto, el grupo, la entidad y los anillos o la marcación que haya bajo el pixel (las marcaciones, encima, ganan a los anillos; se prueba con la misma geometría que el dibujo). La librería no muestra menú propio: cada app decide qué acciones tienen sentido para cada cosa.

Reproducción de historiales. `TrackPlayer` convierte cada pista de un GPX con horas en un **objetivo normal** mientras existió. Usa solo la API pública: `addTarget` / `updateTarget` con los puntos grabados y sus horas, y la posición interpolada por `sampleTrack` . Así traza, origen, alertas de zona y CPA y anillos funcionan solos. Al ir hacia atrás rehace cada objetivo desde su origen. Las rutas completas van en una capa temporal que, como toda capa temporal, queda **fuera de deshacer/rehacer** (`OverlayModel::setLayerUndoable`).

Inicio y fin de rutas. Al leer GPX, cada ruta o track guarda `gpx_kind` y, si sus puntos traen hora, `time_start` y `time_end` . `FeatureLayer` marca su primer punto (verde) y el último (rojo). La línea seleccionada los rotula con la hora, hacia fuera del recorrido para que no se pisen (`setRouteEndpoints`).

Alertas de proximidad. Con `setProximityAlert(distancia, horizonte)` , cada `addTarget` / `updateTarget` pasa también por `ProximityMonitor` , que emite `proximityAlertRaised(a, b, cpa, tcpa)` o `proximityAlertCleared(a, b)` . La velocidad de cada objetivo es la que da la app (`setTargetVelocity` , p. ej. SOG/COG del AIS) o, si no, la que estima `TargetModel` por los últimos 30 s de su historial. Así `MapTarget::speed` sigue sin interpretarse. `TargetLayer` dibuja cada par en alerta encima de los objetivos: discontinuas hasta el punto de CPA de cada uno, la distancia mínima entre ellos y el rótulo «CPA 0.30 NM en 4 min».

9.3 · Entidades y ficheros .geo

`loadGeoAsLayer(ruta)` → `GeoFile` lee los trazados → `OverlayModel` crea una entidad multi-parte → `FeatureLayer` la pinta. `saveFeaturesTo()` vuelca todo con `VectorRepository` .

10 · Codificación de teselas (la matemática)

Cada BD guarda las teselas con su propio convenio. La librería lo captura en el `TileDataset` y traduce siempre igual. Dado un **tesela lógica** `(z, x, y)` en el estándar XYZ (origen arriba-izquierda):

```
z_guardado = zFactor · z + zOffset        (columna z de la tabla)
x_guardado = x                            (igual)
y_guardado = toStorageY(y, z, esquema)    XYZ: y   |   TMS: (2^z − 1) − y
s          = sValue                        (si la BD tiene columna 's')
```

BD	zFactor	zOffset	esquema	columna s	Ejemplo z lógico 11
OSM (Cuba)	1	1	XYZ	sí (2686452)	z_guardado = 12
Satelital (Google)	−1	17	XYZ	sí (0)	z_guardado = 6
Vector / base nueva	1	0	XYZ	no	z_guardado = 11

Este mapeo es **el mismo** que usan al leer el widget y al escribir las herramientas de descarga. Por eso una tesela bajada por `fill_tiles` aparece exactamente donde el mapa la espera.

11 · Herramientas desarrolladas

Programas de línea de comandos (y uno con mapa) que se apoyan en la librería.

Herramienta	Tipo	Para qué
`probe_db`	consola	Sonda una `.sqlitedb` y deduce su configuración; imprime el bloque para `datasets.json`.
`geo_to_tiles`	consola (Gui)	Rasteriza un vector `.geo/.xyz` a una pirámide de teselas SQLite (capa base rápida).
`render_map`	utilidad (widget)	Renderiza el mapa a imagen sin interacción (pruebas/verificación).
`vector_tile`	consola (Gui)	Inspecciona un MBTiles vectorial (capas, claves, clases), pinta una vista a PNG con el estilo por defecto o uno propio y vuelca el estilo por defecto.
`bench_tiles`	consola	Mide el rendimiento de lectura/caché de teselas.
`fill_tiles`	consola	Descarga las teselas que faltan (o crea una base nueva) de una fuente XYZ sin clave.
`fill_map`	ventana + mapa	Lo mismo pero visual: marca el área en el mapa, rango de zoom, barra de progreso.
`fill_hgt`	consola	Descarga `.hgt` de 30 m de AWS Skadi (sin clave) para una zona (`--bbox` / `--cuba`); descomprime gzip con **miniz** vendoreado (sin zlib externa); reanudable. Alimenta a `dem_to_db`.
`dem_to_db`	consola	Construye la base de datos de elevación (`.sqlitedb`) desde una carpeta de `.hgt` (blobs `qCompress` , ~30 % del tamaño). La lee `SqliteElevation`.

Pipeline de elevación empaquetable: `fill_hgt --cuba` (baja 30 m sin clave) → `dem_to_db` (BD comprimida) → `fill_map --dem-db` o `MapWidget::elevationAt()` en tu app. El generador acepta también `.hgt` de 90 m ya existentes.

A continuación, la **síntesis de comando y los argumentos de entrada de cada herramienta**. Las opciones entre `[…]` son opcionales; el resto, obligatorias. Los bbox siempre son `latN,lonO,latS,lonE` (norte, oeste, sur, este).

probe_db — sondar BD y generar datasets.json / mapa.json

```
probe_db --id <id> --file <ruta.sqlitedb> [--id … --file …] [--ref-bbox latN,lonO,latS,lonE] [--out datasets.json] [--no-test] [--package [--dem ruta] [--overlay f.geo]… [--features nombre]]
```

`--id`	Identificador de la capa. **Repetible**; en el mismo orden que los `--file`.
`--file`	Ruta al `.sqlitedb` a sondar. **Repetible**, pareado con `--id`.
`--name`	Nombre visible de la capa (opcional, repetible).
`--ref-bbox`	Rectángulo de referencia para desempatar esquema XYZ vs TMS. Cuba: `23.3,-85.0,19.7,-74.0`.
`--out`	Fichero JSON de salida (def. `datasets.json`). Las rutas se escriben **relativas a su carpeta**.

`--no-test`	No verificar que la fuente lee de verdad (solo deduce).
`--package`	Genera el manifiesto completo de paquete (`mapa.json` v2): `package` (id y nombre de la carpeta, fecha de hoy, zona del `--ref-bbox`), `start` y `features` .
`--dem`	Con `--package` : BD de elevación o carpeta de `.hgt` .
`--overlay`	Con `--package` : capa fija `.geo` (repetible).
`--features`	Con `--package` : nombre de la BD de entidades del usuario (def. `entidades.db`).

check_data — comprobar un paquete antes de distribuirlo

`check_data <carpeta_paquete | mapa.json> [--quick] [--max-zoom N] [--strict]`

`paquete`	Carpeta del paquete o su `mapa.json` .
`--quick`	Solo lo rápido (ficheros, apertura, decodificación); sin cobertura por zoom.
`--max-zoom`	No medir cobertura por encima de este zoom.
`--strict`	Salir con 1 también si hay avisos (por defecto solo con errores).
`--export`	Si no hay errores, copia el `mapa.json` y solo los ficheros que referencia a esa carpeta (reanudable) y comprueba la copia.
`--verbose`	Muestra también el registro interno de la librería.

Imprime una ficha por capa (tamaño, formato de imagen y cobertura por zoom dentro de la zona) y los hallazgos de más a menos graves. Salida 0 = listo para distribuir, 1 = errores, 2 = argumentos mal; pensado para scripts.

geo_to_tiles — rasterizar vector a teselas

`geo_to_tiles --in <fichero.geo> --out <salida.sqlitedb> --id <id> --name "<nombre>" [--minzoom N] [--maxzoom N] [--color #hex] [--width f] [--fill] [--bg #hex]`

`--in`	Fichero de entrada `.geo` / `.xyz` (trazados separados por `0,0`).
`--out`	`.sqlitedb` de teselas a crear.
`--id / --name`	Identificador y nombre de la capa generada.
`--minzoom / --maxzoom`	Rango de zoom de la pirámide (def. 4..11).
`--color / --width`	Color y grosor de línea del trazado.
`--fill / --bg`	Rellenar polígonos / color de fondo.

render_map — mapa a imagen (sin ventana)

`render_map (--datasets <datasets.json> | --data <carpeta_paquete>) --out <mapa.png> [--layer id] [--center lat,lon] [--zoom N] [--size AnchoxAlto] [--wait ms] [--grid] [--features fichero.geo]`

`--datasets / --out`	Config de capas y PNG de salida (obligatorios).
`--data`	En vez de `--datasets` : abre un paquete (`mapa.json`); capa, centro y zoom salen de su `start` salvo que se pasen, y dibuja sus capas fijas.

`--layer`	Capa base a dibujar (def. la primera del JSON).
`--center / --zoom`	Centro `lat,lon` y nivel de zoom.
`--size`	Tamaño en píxeles, `1280x800`.
`--wait`	Milisegundos de espera a que terminen de cargar las teselas.
`--grid`	Dibuja la rejilla `z/x/y` (verde = propia, rojo = respaldo escalado).
`--features`	Superpone las entidades de un `.geo`.

vector_tile — inspeccionar y pintar un MBTiles vectorial

`vector_tile --mbtiles <f.mbtiles> (--info | --out <vista.png>) [--center lat,lon] [--zoom N] [--size AnchoxAlto] [--style-file estilo.json] · vector_tile --print-style`

`--mbtiles`	MBTiles vectorial (`format = pbf`).
`--info`	Capas, nº de entidades por tipo, claves y valores de `class` de la tesela del centro.
`--out`	Pinta la vista (mosaico de teselas de 256 px) a PNG y dice cuánto tardó cada tesela.
`--center / --zoom / --size`	Centro `lat,lon` (def. La Habana), zoom (def. 14) y tamaño (def. `1024x768`).
`--style-file`	Estilo JSON propio en vez del de OpenMapTiles (no `--style`: Qt se queda con esa opción).
`--land-mask / --land-layer`	Máscara de tierra (BD de costa) y su capa: pinta el mar.
`--hide`	Grupos del estilo que no se pintan (`nombres_calles,limites,…`; `--info` los lista).
`--hybrid`	Estilo del mapa híbrido (PNG con fondo transparente); con `--print-style`, lo vuelca.
`--print-style`	Vuelca el estilo por defecto, plantilla para uno propio.

Necesita `QGuiApplication` para los rótulos: sin pantalla, `QT_QPA_PLATFORM=offscreen`.

bench_tiles — medir rendimiento de lectura

`bench_tiles --datasets <datasets.json> [--zoom N] [--width N] [--height N] [--center lat,lon] [--pans N]`

`--datasets`	Config de capas a medir (obligatorio).
`--zoom`	Nivel de zoom de la prueba.
`--width / --height`	Tamaño del viewport simulado.
`--center`	Centro `lat,lon` del barrido.
`--pans`	Número de desplazamientos a simular.

fill_hgt — descargar elevación SRTM (30 m)

`fill_hgt (--cuba | --bbox latN,lonO,latS,lonE) --out <carpeta> [--url base] [--res 30]`

`--cuba`	Preset: bbox de Cuba (lat 19..24, lon −85..−74).

`--bbox`	Zona a cubrir, `latN,lonO,latS,lonE` (alternativa a `--cuba`).
`--out`	Carpeta donde dejar los `.hgt` (reanudable: salta los que ya están).
`--url`	Base de la fuente (def. AWS Skadi `elevation-tiles-prod/skadi`).
`--res`	Resolución; hoy solo **30** m (Skadi). Se acepta y se ignora.

dem_to_db — carpeta .hgt → base de datos de elevación

`dem_to_db <carpeta_hgt> --out <dem.sqlitedb> [--overwrite]`

`<carpeta_hgt>`	Carpeta con los `.hgt` (30 m o 90 m; la resolución se autodetecta por tile). Argumento posicional.
`--out`	Base de datos `.sqlitedb` a crear (tablas `dem_tiles + dem_meta`, blobs `qCompress`).
`--overwrite`	Rehacer la BD si ya existe.

fill_tiles — argumentos

`--datasets / --id`	Rellenar una BD existente: de `datasets.json` toma codificación y ruta de la capa.
`--new / --name`	Crear una base nueva desde cero (codificación limpia XYZ, sin columna s).
`--bbox latN,lonO,latS,lonE`	Rectángulo a cubrir.
`--poly "lat,lon;lat,lon;…"`	Polígono (≥3 vértices): solo baja las teselas cuyo centro cae dentro. Alternativa a --bbox.
`--minzoom / --maxzoom`	Rango de zoom.
`--url`	Fuente XYZ (por defecto Esri «Clarity», sin clave).
`--rate / --conns`	Ritmo (lanzamientos/seg, tope suave) y peticiones en vuelo a la vez (1..8, def. 2).
`--retries / --timeout`	Reintentos por tesela y espera máxima por petición.
`--overwrite / --only-missing`	Rebajar todo / (def.) solo lo que falta.
`--yes`	No preguntar confirmación.

fill_map — cómo se usa

`fill_map [datasets.json] [--dem <carpeta_hgt>] [--dem-db <dem.sqlitedb>]`

El primer argumento posicional es el `datasets.json` (def. `./datasets.json`); `--dem` fija una carpeta de `.hgt` y `--dem-db` una base de datos de elevación. Todo lo demás se maneja desde la ventana:

- **Rellenar**: marca un rectángulo (o escribe el bbox) + rango de zoom → completa la capa activa.
- **Polígono**: marca un polígono clic a clic (doble clic / Enter lo cierra) y descarga solo las teselas de dentro. Excluyente con "Seleccionar área".
- **Nueva base…**: crea un `.sqlitedb` nuevo, todo de la fuente elegida, del área marcada.
- **Estimación de tamaño**: antes de confirmar, un muestreo rápido (`estimateSize`) dice cuántos MB ocupará, además del nº de teselas. Si no logra muestrear, da una cifra "aprox.".

- **Descargar al navegar**: mientras te mueves, baja lo que falte al zoom actual.
- Tres formas de indicar el área: dibujar rectángulo, escribir `bbox`, o la vista actual.
- **Rejilla**: dibuja el borde de cada tesela con su `z/x/y` (verde = propia, rojo = respaldo escalado) para ver qué falta a este zoom.
- **Cobertura** (+ selector de zoom): mancha fija que muestra qué zonas de ESE zoom ya están en la BD, visible aunque mires a un zoom menor; se refresca sola mientras descargas (ver `CoverageLayer`).
- **DEM…** (o `--dem <carpeta>`): carga ficheros SRTM `.hgt` y muestra la **cota del terreno bajo el cursor** en la barra de estado; "—" si no hay dato (ver módulo `dem`).

Orden de descarga (`TileFiller::advanceCursor`): por nivel de zoom de menor a mayor (termina un nivel antes de pasar al siguiente, emitiendo `zoomFinished`); dentro de cada nivel, columna a columna de oeste a este y cada columna de norte a sur; saltando las teselas que ya existen (salvo `--overwrite`). Es un barrido sistemático del bbox, reanudable.

12 · Motor de descarga `TileFiller`

El corazón común de `fill_tiles` y `fill_map` (`tools/common/TileFiller.h`). Es un `QObject` **asíncrono**: no bloquea, se apoya en el bucle de eventos (un temporizador marca el ritmo y cada respuesta encadena la siguiente). Así el mismo motor —y la misma codificación probada— sirve para la consola y para la ventana con barra de progreso viva.

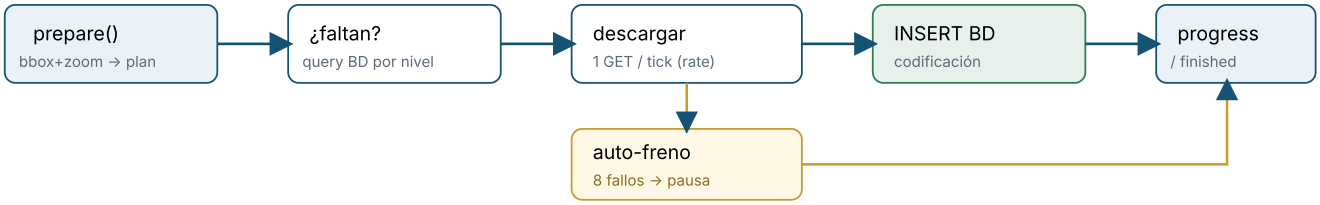


Figura 3 — Flujo de `TileFiller`: estima, descarga a ritmo, inserta con la codificación de la base y avisa; si hay muchos fallos seguidos, pausa y reanuda.

Función / señal	Qué hace
`prepare(params, err)`	Abre la BD (la crea si `createSchema`), calcula cuántas faltan por zoom. Sin red.
`estimateSize(n=12)`	Muestreo asíncrono: baja `n` teselas repartidas por zoom (solo mide bytes, no escribe BD) y emite `sizeEstimated`. Entre `prepare()` y `start()`.
`signal sizeEstimated(KiB, bytesTotal, muestras)`	Tamaño estimado ≈ media·total; `muestras=0` = heurística (sin red) ~20 KiB/tesela.
`start() / cancel()`	Empieza / corta la descarga asíncrona.
`signal progress(hechas,total,t/s)`	Avance y velocidad media (barra + ETA).
`signal zoomFinished(z, nuevas)`	Un nivel terminado.
`signal throttling(seg, racha)`	**Auto-freno**: la fuente limita; pausa `seg` y reanuda.
`signal message / finished`	Motivo real de un fallo / fin con estadísticas.

Propiedades clave: reanudable solo baja lo que falta (se corta y se relanza sin repetir); paralelo hasta `connections` peticiones en vuelo a la vez (cada una con su `Pending` y su timeout), con `rate` como tope de lanzamientos/seg; codificación exacta inserta con `storedZ` /Y/s de la base; polígono descarta las teselas con centro fuera; auto-freno pausa con backoff creciente (30→60→120… →300 s) ante rachas de fallos y reanuda al primer éxito; busy_timeout convive con el mapa leyendo la misma BD.

Interacción con el ratón

- **El clic se decide al soltar.** Pulsar solo agarra etiquetas y entidades `draggable`; el desplazamiento empieza al superar 4 px, así que pulsar sobre un objetivo ya no impide mover el mapa.
- **Doble clic** = pulsación + zoom +1 en vacío. **Rueda** acumulada. **Botón central** desplaza con cualquier herramienta.

- **Señales:** `elementClicked(MapHit, QPoint)` (tarjeta), `contextMenuRequested` (menú), `pickCancelled()`, `interactionBlocked(QString)` (capa bloqueada).
- **Índice por capa** en `OverlayModel` (`forEachInLayer`): la búsqueda bajo el cursor no copia ni ordena todas las entidades.

13 · Decisiones de diseño e historia

- **Núcleo sin ventanas.** Separar `libmapa_core` de `libmapa_widget` permite probar el núcleo sin GUI (14 de los 17 tests) y no imponer QCustomPlot.
- **PIMPL en MapWidget.** QCustomPlot queda dentro; la cabecera pública es limpia.
- **Configuración en datos.** El mapeo de zoom, el esquema y las columnas viven en `datasets.json`, no cableados en el código.
- **Zoom hasta z15+.** `recommendedMaxZoom` separa «hasta dónde hay cobertura buena» de «hasta dónde hay teselas sueltas».
- **Compatibilidad Qt.** Objetivo actual **Qt 5.14 / 5.15 / 6.x**; el código es C++17 y usa guardas `QT_VERSION` solo donde Qt5 y Qt6 difieren de verdad (redirecciones de red, `qHash`, `QWheelEvent::position`).
- **Fuente por defecto Esri «Clarity».** Comparada contra la base Google, Clarity es la fuente gratis sin clave que más se acerca en color (menos costuras al rellenar huecos).
- **Descarga responsable.** Ritmo configurable y **auto-freno**: los servicios gratis limitan el uso masivo; conviene z altos solo por zonas y ritmo suave.

14 · Cómo compilar y usar

CMake (recomendado)

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
# Windows/MinGW:  cmake -G "MinGW Makefiles" -S . -B build ...
```

El widget (MapWidget, fill_map, demo) requiere **QCustomPlot** en `third_party/qcustomplot/` (gitignored). Sin él, se compila solo el núcleo y las herramientas de consola, y CMake lo avisa.

Usarla desde otra aplicación

```
cmake --install build --prefix C:/libmapa
# en la app:
find_package(libmapa REQUIRED)        # CMAKE_PREFIX_PATH = C:/libmapa;<Qt>
target_link_libraries(miapp PRIVATE libmapa::widget)
```

Bibliotecas estáticas, solo las cabeceras públicas, `libmapaConfig.cmake` (busca las mismas dependencias de Qt) y las herramientas. Plantilla: `examples/app_minima`; el API de objetivos (alertas, anillos, filtro, agrupación, reproducción y exportación) en `examples/app_seguimiento`. Una guarda en CMake impide que la librería enlace `Qt Network`: el producto trabaja sin conexión.

Llevarla a otro PC

```
herramientas\desplegar.bat <app.exe> <destino> [carpeta del paquete]
```

`windeployqt` sin los plugins de red, comprobación de `qwindows` / `qsqlite` / `qjpeg`, y `check_data --export`, que copia solo los ficheros del paquete a `<destino>\datos`. La carpeta resultante funciona sin Qt ni internet.

Qt 5.14 y qmake: el segundo juego de despliegue

Para quien no tiene CMake o compila con Qt 5, la librería también se compila e instala con qmake (Release y Debug), en una carpeta por versión de Qt:

```
qmake qmake/libmapa/libmapa.pro  &&  mingw32-make  &&  mingw32-make install
#  -> C:/libmapa/qt5  o  C:/libmapa/qt6
# en el .pro de la app (elige sola la instalación del Qt del kit):
include(C:/libmapa/qt$${QT_MAJOR_VERSION}/libmapa.pri)
```

`libmapa.pri` se para si la app y la librería son de Qt distintos. Para desplegar una app de Qt 5 se usa `desplegar_qt5.bat`, con los mismos argumentos. Comprueba que el `.exe` es de Qt 5 y está en Release, quita a mano los plugins de red (el `windeployqt` de Qt 5 no tiene `--skip-plugin-types`) y avisa si algo sigue pidiendo `Qt5Network.dll`. Guía paso a paso para alguien sin experiencia: `docs/DESPLIEGUE.md`.

```
qmake qmake/fill_tiles.pro  &&  make      # herramientas sueltas: fill_map.pro / probe_db.pro
```

Un flujo típico de datos

1. Preparar los datos (con internet): `fill_tiles` / `fill_map` para teselas, `fill_hgt` → `dem_to_db` para la elevación.
2. `probe_db --package` sobre la carpeta → genera su `mapa.json` (rutas relativas). Se ajusta a mano lo que haga falta (nombres, atribución, estilos).
3. `check_data` sobre la carpeta: 0 errores → el paquete está listo.
4. `desplegar.bat` (Qt 6) o `desplegar_qt5.bat` (Qt 5) con la app y el paquete → carpeta lista para otro PC.
5. Se copia la carpeta junto a la aplicación. La app solo necesita `cfg.dataDir = …/datos`: el producto final **no usa internet**.
